

explain me datetime and os modules for build files and save it on the disk over specified time

Absolutely. Since you're learning Python to **generate JSON data for Spark Structured Streaming and Kafka**, `datetime` and `os` are two very useful modules.

Think of them like this:

- `datetime` → **What time is it?**
- `os` → **Where should I create/save the file?**
- Together → **Create files with timestamps and save them into folders on disk.**

1. The `datetime` module

Python's `datetime` module lets you work with **dates and times**.

Python

```
from datetime import datetime
```



Run

Now:

Python

```
now = datetime.now()
```

```
print(now)
```



Run

Example output:

```
2026-08-29 20:18:32.451234
```



It contains:

```
2026 - 08 - 29    20 : 18 : 32
   date           time
```



2. Get individual parts

</> Python



▶ Run

```
from datetime import datetime

now = datetime.now()

print(now.year)
print(now.month)
print(now.day)

print(now.hour)
print(now.minute)
print(now.second)
```

Example:

```
2026
8
29

20
18
32
```



This is useful when creating files based on time.

3. Format datetime

Usually you don't want the full datetime.

You can format it using `strftime()` .

</> Python



▶ Run

```
from datetime import datetime

now = datetime.now()

formatted = now.strftime("%Y-%m-%d")

print(formatted)
```

Output:

2026-08-29



Common formats:

Code	Meaning	Example
%Y	Year	2026
%m	Month	08
%d	Day	29
%H	Hour	20
%M	Minute	18
%S	Second	32

For example:

</> Python



▶ Run

```
timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")  
  
print(timestamp)
```

Output:

2026-08-29_20-18-32



This is excellent for filenames.

For example:

events_2026-08-29_20-18-32.json



4. The os module

os allows Python to interact with the operating system.

</> Python



▶ Run

```
import os
```

For example, get the current directory:

```
</> Python  
print(os.getcwd())
```



Run

You might get:

```
/home/youssef/python-project
```



5. Create a directory

Suppose you want:

```
data/
```



You can do:

```
</> Python  
import os  
os.mkdir("data")
```



Run

Now:

```
python-project/  
|  
├─ data/  
└─ script.py
```



But there's a problem.

If `data` already exists:

```
</> Python  
os.mkdir("data")
```



Run

will produce an error.

A better approach is:

</> Python



Run

```
os.makedirs("data", exist_ok=True)
```

Now Python will create the directory **if it doesn't exist**.

If it already exists, nothing happens.

6. Create nested directories

You can create:

```
data/  
├── json/  
│   └── events/
```



with:

</> Python



Run

```
import os  
  
os.makedirs("data/json/events", exist_ok=True)
```

7. Combine datetime + os

Now we can start doing something useful.

Suppose you want to create a JSON file containing an event.

</> Python



Run

```
import os
import json
from datetime import datetime

os.makedirs("data", exist_ok=True)

timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")

filename = f"event_{timestamp}.json"

filepath = os.path.join("data", filename)

event = {
    "id": 1,
    "product": "Laptop",
    "price": 1000,
    "timestamp": datetime.now().isoformat()
}

with open(filepath, "w") as file:
    json.dump(event, file, indent=4)

print("File created:", filepath)
```

You might get:

```
data/event_2026-08-29_20-25-10.json
```



And the file contains:

```
</> JSON

{
  "id": 1,
  "product": "Laptop",
  "price": 1000,
  "timestamp": "2026-08-29T20:25:10.123456"
}
```



8. Why use `os.path.join()` ?

Instead of:

```
</> Python

filepath = "data/" + filename
```



▶ Run

use:

</> Python



▶ Run

```
filepath = os.path.join("data", filename)
```

Python handles the appropriate path separator for the operating system.

For example:

</> Python



▶ Run

```
folder = "data"
filename = "event.json"

filepath = os.path.join(folder, filename)

print(filepath)
```

Linux:

data/event.json



9. Creating files over time

This is especially important for **Spark Structured Streaming**.

Imagine you want Python to continuously generate JSON files:

```
data/
├─ event_20-00-01.json
├─ event_20-00-05.json
├─ event_20-00-09.json
├─ event_20-00-13.json
└─ ...
```



Spark can watch the directory:

</> Python



▶ Run

```
spark.readStream \
  .format("json") \
  .schema(schema) \
  .load("data/")
```

Whenever a **new JSON file** appears, Spark can process it.

10. `time.sleep()`

`datetime` tells us the current time, but it doesn't make Python wait.

For waiting, use the `time` module.

</> Python



▶ Run

```
import time

print("Start")

time.sleep(5)

print("After 5 seconds")
```

The program waits 5 seconds.

So:

```
datetime → know the time
time.sleep → wait
os → work with directories/files
json → create JSON
```



These modules work very well together.

11. Generate a JSON file every 5 seconds

Here's a simple streaming-data generator:

</> Python



▶ Run


```
import os
import json
import time
from datetime import datetime

folder = "data"

os.makedirs(folder, exist_ok=True)

counter = 1

while True:

    event = {
        "id": counter,
        "product": "Laptop",
        "price": 1000,
        "timestamp": datetime.now().isoformat()
    }

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")

    filename = f"event_{timestamp}_{counter}.json"

    filepath = os.path.join(folder, filename)

    with open(filepath, "w") as file:
        json.dump(event, file, indent=4)

    print("Created:", filepath)

    counter += 1

    time.sleep(5)
```

It will continuously produce:

```
data/
|
├─ event_2026-08-29_20-30-01_1.json
├─ event_2026-08-29_20-30-06_2.json
├─ event_2026-08-29_20-30-11_3.json
├─ event_2026-08-29_20-30-16_4.json
└─ ...
```



Every 5 seconds.

12. Very important: datetime vs time

You should understand the difference.

`datetime`

Used for representing dates and times:

</> Python



▶ Run

```
from datetime import datetime

now = datetime.now()

print(now)
```

`time`

Useful for delays:

</> Python



▶ Run

```
import time

time.sleep(5)
```

So this:

</> Python



▶ Run

```
datetime.now()
```

means:

Tell me the current date/time.

While:

</> Python



▶ Run

```
time.sleep(5)
```

means:

Stop the program for 5 seconds.

13. Creating a file every minute

Change:

```
</> Python
```



▶ Run

```
time.sleep(5)
```

to:

```
</> Python
```



▶ Run

```
time.sleep(60)
```

Now:

```
create file
  ↓
wait 60 seconds
  ↓
create file
  ↓
wait 60 seconds
  ↓
create file
```



14. Stop the program

Because we used:

```
</> Python
```



▶ Run

```
while True:
```

the program never stops by itself.

You can stop it in the terminal with:

```
ctrl + c
```



In Jupyter, interrupt the kernel or stop the running cell.

15. One important improvement for Spark

For Spark Structured Streaming, you usually **don't need one JSON object per file**.

You can generate multiple events into separate files, for example:

```
data/  
├─ batch_001.json  
├─ batch_002.json  
├─ batch_003.json  
└─ ...
```



Each file could contain:

```
</> JSON  
  
{ "id": 1, "product": "Laptop", "price": 1000 }  
{ "id": 2, "product": "Mouse", "price": 50 }  
{ "id": 3, "product": "Keyboard", "price": 100 }
```



This is called **JSON Lines / JSONL**, and it is particularly convenient for Spark file streaming.

The mental model you should learn

For your Spark/Kafka practice, remember this pipeline:

Python

- datetime → generate timestamps
- json → create JSON data
- os → create folders + file paths
- time → wait between files

▼

JSON files on disk

▼

Spark Structured Streaming

▼

Transformations

▼

HDFS / Parquet / Delta

And later, when you learn Kafka:

Python

- datetime
- json
- kafka-python / Kafka client

▼

Kafka

▼

Spark Streaming

Next, the most useful exercise is to build a Python script that generates realistic orders JSON files every 5 seconds using datetime + os + json + random , then connect that folder directly to your Spark Structured Streaming code.

📄 👍 📤 ↻ ...

Do you like this personality? 👍 👎 ✕

ChatGPT can make mistakes. Check important info.

